

B.Sc. (Data Science)

DS101T : Problem Solving and Python Programming

Unit 3: Classes, Objects and Inheritance

Mrs. Reshma Masurekar
Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18

Classes, Objects and Inheritance

- **Classes and Objects**
- Classes as User Defined Data Type
- Objects as Instances of Classes
- Creating Class and Objects
- Creating Objects By Passing Values
- Variables & Methods in a Class

- **Inheritance**
- Single Inheritance
- Multilevel
- Multiple Inheritance
- Hybrid Inheritance
- Hierarchical Inheritance
- IS-A Relationship and HAS-A Relationship

Object-oriented programming principles

- Python is an object oriented programming language.
- Major principles of object-oriented programming system are given below:
 - Class
 - Object
 - Method
 - Inheritance
 - Polymorphism
 - Data Abstraction
 - Encapsulation

Classes

- In Python, a class is a *user-defined data type* that contains both the *data itself and the methods* that may be used to manipulate it.
- A class is a *collection of objects*.
- A class contains the blueprints or the prototype from which the objects are being created.
- It is a logical entity that has some specific attributes and methods

Classes as User Defined Data Type

- In Python, a **class can be created by using the keyword `class`, followed by the class name.**
- **Syntax:**
`class ClassName:`
 `#statement_suite`
- A class contains a statement suite including fields, constructor, function, etc. definition.

Objects as Instances of Classes

- An object is an instance of a class that encapsulates both data and behavior related to that data. Objects are the fundamental building blocks of OOP, and they can represent real-world entities or abstract concepts.
- An object consists of:
 - **Attributes:** These are the data or state of the object, typically represented by variables. Each object can have its own values for these attributes.
 - **Methods:** These are the behaviours or functions that the object can perform. Methods define the operations that can be performed on the object's data.
 - **Identity:** Each object has a unique identity, distinguishing it from other objects, even if they have the same attributes and methods
- **syntax to create an object:**
`objectName = ClassName()`

Access Class Attributes Using Objects

- All the instances share the attributes and the behavior of the class. But the values of those attributes, i.e. the state are unique for each object. A single class may have any number of instances.
- **create multiple objects from a single class:**
 objectName1 = ClassName()
 objectName2 = ClassName()
- **Attributes and functions** of the class are always public and can be accessed using the **dot (.)** operator.

Demonstration of class and object

```
class Number:
```

```
    x=10
```

```
n1=Number()
```

```
print(n1.x)
```


method

- A Python function defined inside a class is called a **method**.
- Objects can also contain methods. Methods in objects are functions that belong to the object.

```
# create a class
class Room:
    length = 0.0
    breadth = 0.0

    # method to calculate area
    def calculate_area(self):
        print("Area of Room =", self.length * self.breadth)

# create object of Room class
study_room = Room()

# assign values to all the properties
study_room.length = 42.5
study_room.breadth = 30.8

# access method inside class
study_room.calculate_area()
```

The self-parameter

- The self parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.
- We can use anything instead of self, but it must be the first parameter of any function which belongs to the class.

```
class Employee:
    def accept(self):
        self.eid=int(input("Enter Employee id:"))
        self.ename=input("Enter Employee name:")
        self.esalary=int(input("Enter Employee salary:"))
    def display(self):
        print("Employee id: " ,self.eid)
        print("Employee name: " ,self.ename)
        print("Employee salary: " ,self.esalary)
```

```
e1=Employee()
e1.accept()
e1.display()
```

`__init__()`: Creating Objects By Passing Values

- All classes have a function called `__init__()`, which is always executed when the class is being initiated.
- Use the `__init__()` function to assign values to object properties, or other operations that are necessary to do when the object is being created.
- **Note:** The `__init__()` function is called automatically every time the class is being used to create a new object.

#Creating Objects By Passing Values

class Person:

def __init__(self, name, age):

self.name = name

self.age = age

p1 = Person("John", 36)

print(p1.name)

print(p1.age)

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.display()
```